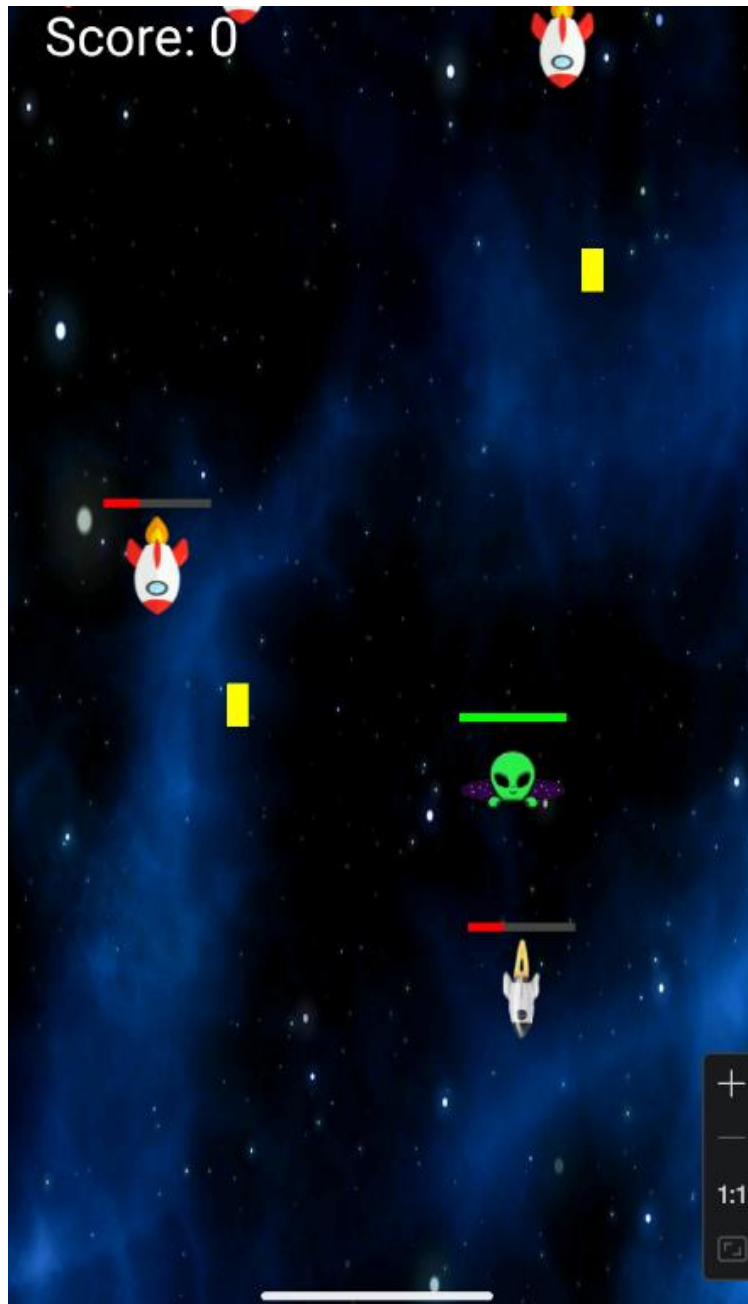


Lab 4: Real-Time Mobile Application Development

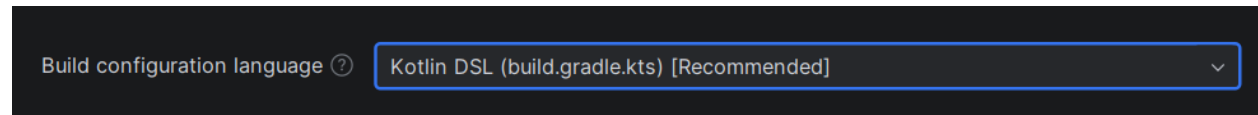
In this lab, we will build a simple game similar to the classic chicken shooting game.



For this assignment, each group must upload their source code to GitHub and submit a PDF report. For every exercise in the assignment, the report must include the corresponding code and an explanation of the processing logic.

Resources such as images, themes, colors, and source code are provided in the project folder.

Note: This assignment uses Kotlin. Therefore, after selecting **Empty Activity**, you must choose **Kotlin DSL** in the **Build configuration language** section.



I. Instructions

1. File `GameThread.kt`

The `GameThread.kt` file is responsible for creating the main game loop. In a 2D game, the screen must be continuously updated to create the feeling of movement. If the game is drawn only once, enemies, bullets, and the background will remain static. Therefore, a separate thread is needed to run continuously alongside the Android UI thread in order to process the game. This thread repeatedly calls the update and draw functions approximately 60 times per second.

Class Declaration

```
class GameThread(  
    private val surfaceHolder: SurfaceHolder,  
    private val gameView: GameView  
) : Thread()
```

This class extends `Thread`, meaning it can run independently from Android's main UI thread. `surfaceHolder` is used to manage the drawing area of the `SurfaceView`, while `gameView` contains all game logic so the thread can call `update()` and `draw()`.

Running Variable

```
var running: Boolean = false
```

This variable controls the game loop. When `running = true`, the game loop continuously executes. When `running = false`, the loop stops and the game ends. This is how the thread is started and stopped.

FPS Variable

```
private val targetFPS = 60
```

FPS (Frames Per Second) is the number of frames displayed each second. A value of 60 FPS means the game attempts to update and redraw the screen 60 times every second to ensure smooth movement. Low FPS causes lag, while very high FPS increases CPU usage and heats up the device.

run() Function

```
override fun run()
```

This is the most important function of *GameThread*. When the thread starts, Android automatically calls this function. The entire game loop is located inside *run()*.

Calculating Frame Time

```
val frameTime = (1000 / targetFPS).toLong()
```

This line calculates the maximum time allowed for one frame. Since one second equals 1000 milliseconds, running at 60 FPS means each frame should take about 16ms. This helps maintain stable game speed.

Main Game Loop

```
while (running)
```

This is the main game loop. While the game is running, the loop continuously:

1. updates game logic,
2. redraws the screen,
3. waits for a short period,
4. repeats again.

Thanks to this loop, enemies can fall, bullets can move upward, and the screen continuously changes.

Measuring Frame Start Time

```
startTime = System.nanoTime()
```

This records the starting time of the current frame. After updating and drawing, the game calculates how long the frame took in order to adjust FPS properly.

Locking the Canvas

```
canvas = surfaceHolder.lockCanvas()
```

Canvas is where all game graphics are drawn. `lockCanvas()` obtains access to the screen before drawing so that multiple threads do not modify the canvas simultaneously.

synchronized(surfaceHolder)

```
synchronized(surfaceHolder)
```

This block ensures only one thread can access `surfaceHolder` at a time, preventing drawing conflicts.

Calling `update()`

```
gameView.update()
```

The `update()` function handles all game logic, including:

- moving enemies,
- updating bullets,
- checking collisions,
- calculating scores,
- spawning new enemies.

This is the “calculation” stage of the game.

Calling `draw()`

```
gameView.draw(canvas)
```

After updating the logic, the game is rendered onto the canvas. All backgrounds, enemies, bullets, and scores are displayed here.

unlockCanvasAndPost()

```
surfaceHolder.unlockCanvasAndPost(canvas)
```

After drawing is complete, the canvas is unlocked and displayed on the screen. Without this step, players would not see the new frame.

Calculating Frame Processing Time

```
timeMillis = (System.nanoTime() - startTime) / 1_000_000
```

This calculates how many milliseconds the current frame required.

Calculating Wait Time

waitTime = frameTime - timeMillis

If a frame finishes too quickly, the thread waits for the remaining time to maintain the target FPS. For example, if rendering takes only 10ms, the thread waits another 6ms.

sleep()

sleep(waitTime)

The thread pauses briefly to:

- reduce CPU usage,
- save battery,
- maintain stable performance.

2. File GameManager.kt

This file manages enemy creation. Instead of repeatedly writing enemy creation code inside *GameView*, the logic is separated into *GameManager* for cleaner and easier maintenance.

Class Declaration

class GameManager(private val context: Context)

Context allows access to Android resources such as images inside the drawable folder.

createOpponent() Function

fun createOpponent(x: Float, y: Float, speed: Float): Opponent

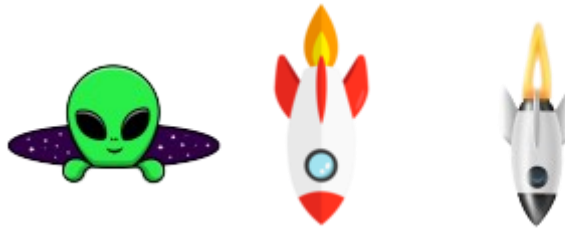
This function creates a new enemy with:

- x position,
- y position,
- movement speed.

Loading Enemy Image

*BitmapFactory.decodeResource(
context.resources,*

```
R.drawable.token_red_removebg  
)
```



This loads an image from the drawable folder and converts it into a Bitmap.

Creating an Opponent Object

```
return Opponent(x, y, speed, customBitmap)
```

After obtaining the bitmap, the game creates an Opponent object containing:

- position,
- speed,
- enemy image.

3. File Opponent.kt

This file represents enemies in the game. Every enemy appearing on the screen is an Opponent object.

Position and Speed Properties

```
var x: Float
```

```
var y: Float
```

```
var speed: Float
```

x and y determine the enemy's position, while speed controls falling speed.

Bitmap

```
private val bitmap: Bitmap
```

Stores the enemy image.

Width and Height

```
val width = bitmap.width.toFloat()
```

```
val height = bitmap.height.toFloat()
```

Gets the actual size of the bitmap.

RectF

```
private val rect: RectF
```

RectF is a rectangle used for:

- drawing the enemy,
- collision detection.

update() Function

```
fun update() {  
    y += speed  
    rect.set(x, y, x + width, y + height)  
}
```

Updates the enemy position every frame. Since the Android Y-axis increases downward, `y += speed` makes the enemy fall.

draw() Function

```
canvas.drawBitmap(bitmap, null, rect, null)
```

This function draws the enemy bitmap onto the screen at the position defined by `rect`. This is the step that displays the enemy for the player to see.

isOffScreen() Function

```
return y > height
```

This function checks whether the enemy has moved outside the screen. If the enemy passes beyond the bottom edge of the screen, the game removes this object to save memory.

getRect() Function

```
fun getRect(): RectF
```

This function returns the enemy's hitbox so the game can check collisions with bullets.

4. File FiringObject.kt

This file manages the player's bullets. Every time the player touches the screen, a new `FiringObject` object is created.

Position and Speed Properties

var x: Float

var y: Float

var speed: Float

x and *y* represent the bullet's position. *speed* is the upward movement speed of the bullet.

Paint

private val paint: Paint = Paint()

Paint is used to configure the bullet's color and drawing style.

Width và Height

val width: Float = 20f

val height: Float = 40f

These values define the bullet's size.

RectF

val rect: RectF

RectF là hitbox collision của bullet.

init

paint.color = Color.YELLOW

Sets the bullet color to yellow when drawing.

update() Function

y -= speed

rect.set(x, y, x + width, y + height)

In Android: decreasing *y* → the object moves upward.

Therefore, the bullet moves toward the top of the screen. After moving, the rectangle (rect) is updated so collision detection matches the bullet's new position.

draw() Function

canvas.drawRect(rect, paint)

This function draws the bullet as a yellow rectangle.

isOffScreen() Function

return y + height < 0

If the bullet moves above the top of the screen, the game removes it from the list to reduce RAM usage and prevent lag.

5. File GameView.kt

GameView.kt is the most important file in the game.

This file acts as the central controller for:

- managing gameplay,
- handling game logic,
- displaying graphics,
- receiving player input,
- controlling enemies and bullets.

surfaceCreated() Function

```
override fun surfaceCreated(holder: SurfaceHolder) {  
    thread.running = true  
    thread.start()  
}
```

This function is called when the *SurfaceView* is ready for drawing on the screen. In Android, the game cannot draw immediately when the app starts because it must wait until the Surface is fully created. At that point, the system calls *surfaceCreated()*.

When the thread starts running:

- the game loop is created,
- *update()* runs continuously,
- *draw()* runs continuously.

surfaceDestroyed() Function

```
override fun surfaceDestroyed(holder: SurfaceHolder) {  
    var retry = true  
    thread.running = false
```

```

while (retry) {
    try {
        thread.join()
        retry = false
    } catch (e: InterruptedException) {
        e.printStackTrace()
    }
}
}

```

This function is called when the SurfaceView is destroyed, for example:

- exiting the game,
- switching screens,
- closing the application.

In the first step, the program sets:

```
thread.running = false
```

This stops the game loop inside *GameThread*.

Then the program uses:

```
thread.join()
```

to wait until the thread completely finishes before closing the application.

This step is very important because if the thread has not stopped before the app closes, it can cause: crashes, memory leaks, SurfaceView errors.

surfaceChanged() Function

```

override fun surfaceChanged(
    holder: SurfaceHolder,
    format: Int,
    width: Int,
    height: Int
) {
    backgroundScaledBitmap =
        Bitmap.createScaledBitmap(

```

```
        backgroundBitmap,  
        width,  
        height,  
        true  
    )  
}
```

This function is called whenever the screen size changes.

Examples:

- rotating the screen,
- resizing the SurfaceView.

In this game, the function is used to scale the background image according to the screen size.

Processing Step

Bitmap.createScaledBitmap(...)

This function:

- enlarges or shrinks the background image,
- helps the background fully cover the screen.

Without scaling:

- the background might appear distorted,
- or it may not fully cover the screen.

update() Function

fun update()

This is the most important gameplay function.

It is called approximately: 60 times per second
by GameThread.

All game logic is handled here:

- enemy movement,

- bullet movement,
- enemy spawning,
- collision detection,
- score calculation,
- game over handling.

Step 1: Check Game Over

if (gameOver) return

If the player has already lost:

- all game logic stops.

This prevents:

- enemies from continuing to fall,
- bullets from continuing to move,
- scores from changing.

The game becomes “frozen” in the losing state.

Step 2: Increase Game Difficulty

*opponentBaseSpeed += score * 0.0001f*

As the score increases, enemy speed gradually increases.

For example:

- low score → enemies move slowly,
- high score → enemies move faster.

Increase Bullet Speed

*firingObjectBaseSpeed += score * 0.0005f*

As the game becomes harder, bullets also become faster.

Speed Limit

if (opponentBaseSpeed > maxSpeed)

Without a limit:

- enemies would become excessively fast,
- the game would become impossible to play.

Therefore, the program sets a maximum speed limit.

Step 3: Update bullet

```
firingObjects.forEach { it.update() }
```

The game loops through all bullets.

Each bullet calls: *FiringObject.update()*

Inside that function: $y -= \text{speed}$ so bullets move upward.

Step 4: Remove Bullets Outside the Screen

```
firingObjects.removeAll {  
    it.isOffScreen(height)  
}
```

After bullets leave the screen, the program removes them from the list.

This is very important because if bullets are not removed:

- RAM usage continuously increases,
- the game becomes laggy,
- the app may crash.

Step 5: Update Enemies

```
opponents.forEach { it.update() }
```

The game loops through all enemies.

Inside *Opponent.update()*: $y += \text{speed}$ so enemies move downward.

Step 6: Remove Enemies Outside the Screen

```
opponents.removeAll {  
    it.isOffScreen(height)  
}
```

If enemies move outside the screen, the program removes them from the game.

Purpose: save memory, reduce lag.

Step 7: Collision Detection

```
if (firingObject.rect.intersect(opponent.getRect()))
```

This is the most important part of gameplay.

The game:

- takes each bullet,
- checks it against each enemy.

If the two rectangles intersect, a collision occurs.

RectF is a rectangle object that helps perform collision detection more efficiently.

When a Collision Happens

```
firingObjects.remove(firingObject)  
opponents.remove(opponent)  
score += 10
```

Results:

- the enemy is destroyed,
- the bullet disappears,
- the score increases.

This is the core gameplay mechanic:

- hit the enemy → gain points.

Why Copy the List?

```
val firingObjectsCopy = ArrayList(firingObjects)
```

If the program removes elements while iterating through the original list, the app may crash.

Therefore, a copy is created for safe iteration.

Step 8: Check Game Over

```
if (opponent.y + opponent.height >= height - 100)
```

If an enemy reaches near the bottom of the screen, the game ends.

This creates the main objective for the player: destroy enemies before they reach the bottom.

Step 9: Spawn New Enemies

```
if (Random.nextFloat() < 0.02)
```

Each frame has a 2% chance to create a new enemy.

This makes enemy appearances random.

Choose a Random Enemy Image

```
val randomBitmap = opponentBitmaps.randomOrNull()
```

The game randomly selects: rocket, alien, spaceship.

Create a New Enemy

```
Opponent(  
    Random.nextFloat() * (width - 80),  
    -80f,  
    opponentBaseSpeed,  
    resizedBitmap  
)
```

Meaning of Each Parameter

x: random horizontal position

-80f: spawn above the screen

speed: falling speed

Bitmap: enemy image

draw() Function

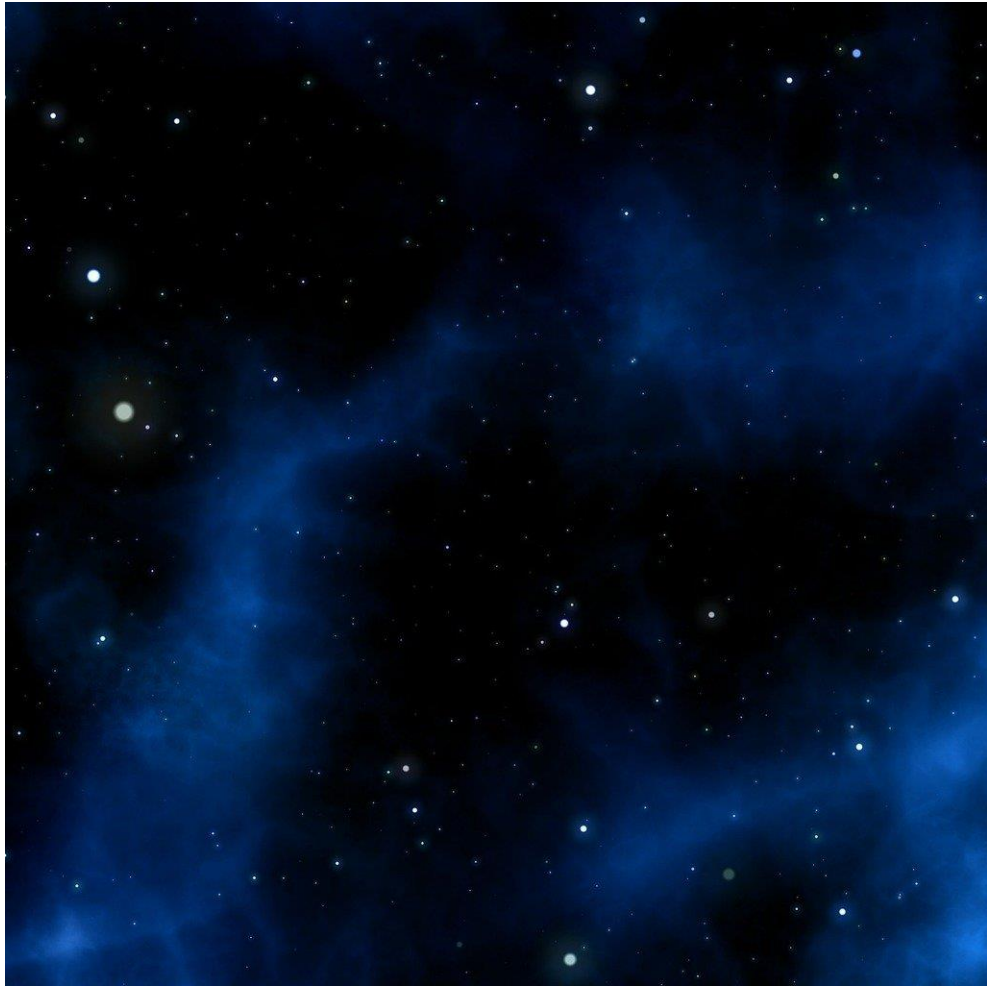
```
override fun draw(canvas: Canvas)
```

This function is responsible for displaying the entire game and is continuously called by *GameThread*.

Step 1: Draw Background

```
drawBackground(canvas)
```

The background is drawn first because it is the bottom layer.



Step 2: Draw Enemies

```
opponents.forEach { it.draw(canvas) }
```

The game loops through all enemies and displays them on the screen.



Step 3: Draw Bullets

```
firingObjects.forEach { it.draw(canvas) }
```

Displays all bullets.

Step 4: Draw Score

```
canvas.drawText("Score: $score")
```


Displays the current score.



Step 5: Draw Game Over

```
if (gameOver)
```

If the player loses, the text “Game Over” is displayed.



Step 6: onTouchEvent() Function

```
override fun onTouchEvent(event: MotionEvent)
```

This function handles player input.

When the Player Touches the Screen

```
if (event.action == MotionEvent.ACTION_DOWN)
```

The game detects that the player has touched the screen.

Create a Bullet

```
val firingObject = FiringObject(
    event.x,
    height - 100f,
    firingObjectBaseSpeed
)
```

The bullet is created: at the touched horizontal position, near the bottom of the screen.



Add Bullet to the Game

firingObjects.add(firingObject)

After being added:

- `update()` moves the bullet,
- `draw()` displays the bullet.

7. resetGame() Function

private fun resetGame()

This function resets the entire game state.

Reset Score

score = 0

The score returns to zero.

Reset speed

opponentBaseSpeed = 5f

The game returns to its initial difficulty.

Remove object

firingObjects.clear()

opponents.clear()

Removes all bullets and enemies.

Restart Game

gameOver = false

Allows the game to continue running.

II. Exercises

1. Create an additional area to display the highest score achieved.
2. Give the player 3 lives for each game session (the player loses only after enemies reach the boundary 3 times).
3. Add a spaceship graphic at the shooting position. Each time the player touches the screen, the spaceship should move to the touched location.
4. Upgrade the bullet system so the bullets can spread/diverge over time. Each upgrade level should fire more bullets.
5. Add a health bar for each enemy. Every enemy should have a random amount of health, and the health bar should gradually decrease whenever the enemy is hit.
6. Make enemies capable of moving across different lanes instead of only moving straight downward.
7. Create a larger alien enemy (Boss) that moves left and right while remaining fixed near the top of the screen. This Boss should spawn smaller aliens from the center of the Boss instead of spawning enemies randomly.